HKUST COMP 4421: Digital Image Processing

Assignment 1 Formal Solutions and Engineering Report

Lin Anfan (Norman) SID: 20861656 Dept. of Mathematics (CS Track)

Fall Semester 2026-27

Course Architecture & Official AI Policy Acknowledgment:
In strict accordance with the HKUST COMP 4421 syllabus regulations (*"Students are free to use generative AI on their works, and should acknowledge it when used"*), the student acknowledges the consultative use of generative AI tools for algebraic consistency validation and formatting assistance. All derived proofs, algorithmic implementations, and analytical conclusions were independently authored, mathematically audited, and verified by the student. A complete and formal disclosure is provided in Appendix **??**.

---

1 Problem 1: Image Arithmetic and Histograms (10 pts)

1.1 Problem Context & Mathematical Lattice Definition

Let the discrete spatial image domain be defined on the two-dimensional lattice:

$$\Omega = \left\{(x, y) \in \mathbb{Z}^2 \;\middle|\; 0 \le x < M,\ 0 \le y < N\right\}, \quad M = 40,\ N = 50. \tag{1}$$

The total spatial cardinality (pixel count) is $|\Omega| = M \times N = 2000$. The pixel intensity range is quantized over a 5-bit discrete integer ring, yielding $L = 2^5 = 32$ quantization levels:

$$\mathbb{Z}_L = \{0, 1, 2, \ldots, L-1\} = \{0, 1, 2, \ldots, 31\}. \tag{2}$$

1.2 (a) Saturated Image Addition Analysis (5 pts)

Image A is formalized as a constant homogeneous field:

$$A(x, y) \equiv 6, \quad \forall (x, y) \in \Omega. \tag{3}$$

Image B is a random scalar field whose discrete probability distribution is specified by its histogram counts $n_B(r) = \#\{(x, y) \in \Omega \mid B(x, y) = r\}$.

The composite image C is constructed via the pointwise non-linear saturation addition operator:

$$C(x, y) = \min\left(A(x, y) + B(x, y),\ 31\right) = \min\left(B(x, y) + 6,\ 31\right). \tag{4}$$

Let $\mathcal{T} : \mathbb{Z}_{32} \to \mathbb{Z}_{32}$ be the forward intensity mapping defined by $\mathcal{T}(r) = \min(r + 6, 31)$. The histogram of C represents the push-forward measure under $\mathcal{T}$:

$$n_C(c) = \sum_{r \in \mathbb{Z}_{32} : \mathcal{T}(r) = c} n_B(r). \tag{5}$$

1.2.1 Step-by-Step Intensity Mapping Derivation

Applying the transformation $\mathcal{T}(r)$ to each non-zero bin of Image B:

1. $r = 0$: $C = \min(0 + 6, 31) = 6$. Transferred pixel count: $n_C(6) = 100$.

2. $r = 2$: $C = \min(2 + 6, 31) = 8$. Transferred pixel count: $n_C(8) = 150$.
3. $r = 5$: $C = \min(5 + 6, 31) = 11$. Transferred pixel count: $n_C(11) = 250$.
4. $r = 10$: $C = \min(10 + 6, 31) = 16$. Transferred pixel count: $n_C(16) = 300$.
5. $r = 15$: $C = \min(15 + 6, 31) = 21$. Transferred pixel count: $n_C(21) = 300$.
6. $r = 20$: $C = \min(20 + 6, 31) = 26$. Transferred pixel count: $n_C(26) = 300$.
7. $r \geq 25$ (Boundary Saturation Regime):

$$r = 25 \implies C = \min(25 + 6, 31) = \min(31, 31) = 31, \quad \text{(count: 250)} \tag{6}$$
$$r = 27 \implies C = \min(27 + 6, 31) = \min(33, 31) = 31, \quad \text{(count: 150)} \tag{7}$$
$$r = 29 \implies C = \min(29 + 6, 31) = \min(35, 31) = 31, \quad \text{(count: 100)} \tag{8}$$
$$r = 31 \implies C = \min(31 + 6, 31) = \min(37, 31) = 31, \quad \text{(count: 100)} \tag{9}$$

The upper dynamic boundary $c = 31$ aggregates all overflowing intensities through a many-to-one mapping:

$$n_C(31) = n_B(25) + n_B(27) + n_B(29) + n_B(31) = 250 + 150 + 100 + 100 = 600. \tag{10}$$

1.2.2 Tabular Results & Conservation Proof

The mapping progression specified by the official assignment template is tabulated in Table **??**, and the resulting non-zero entries of the histogram of C are summarized in Table **??**.

Table 1. Pointwise Saturated Addition Mapping Table for Image C.

B **Intensity** (r)	**Count** (n_B)	**Linear Sum** ($r + 6$)	$C = \min(B + 6, 31)$
0	100	6	**6**
2	150	8	**8**
5	250	11	**11**
10	300	16	**16**
15	300	21	**21**
20	300	26	**26**
25	250	31	**31**
27	150	33	**31**
29	100	35	**31**
31	100	37	**31**

Table 2. Complete Non-Zero Histogram of Image C.

Intensity Level (c)	**Pixel Count** ($n_C(c)$)	**Normalized Probability** ($p_C(c) = n_C/\|\Omega\|$)
6	100	$100/2000 = 0.050$
8	150	$150/2000 = 0.075$
11	250	$250/2000 = 0.125$
16	300	$300/2000 = 0.150$
21	300	$300/2000 = 0.150$
26	300	$300/2000 = 0.150$
31	600	$600/2000 = 0.300$

Measure Conservation Verification:

$$\sum_{c \in \text{supp}(n_C)} n_C(c) = 100 + 150 + 250 + 300 + 300 + 300 + 600 = 2000 = M \times N. \quad \text{(Q.E.D.)} \tag{11}$$

1.3 (b) Difference Imaging and Dynamic Classification (5 pts)

Let $I_{\text{before}}, I_{\text{after}} : \Omega \to \mathbb{Z}_L$ be two co-registered temporal acquisitions. The signed difference image $D(x, y)$ is defined over the difference group $\{-31, \ldots, 31\}$:

$$D(x, y) = I_{\text{after}}(x, y) - I_{\text{before}}(x, y). \tag{12}$$

The spatial domain partitions into three disjoint subsets:

$$\Omega = \Omega_{\text{bright}} \dot{\cup} \Omega_{\text{dark}} \dot{\cup} \Omega_{\text{unchanged}}, \tag{13}$$

where:

$$\Omega_{\text{bright}} = \{(x, y) \in \Omega \mid D(x, y) > 0\} \tag{14}$$
$$\Omega_{\text{dark}} = \{(x, y) \in \Omega \mid D(x, y) < 0\} \tag{15}$$
$$\Omega_{\text{unchanged}} = \{(x, y) \in \Omega \mid D(x, y) = 0\}. \tag{16}$$

1.3.1 Dynamic Event Analysis

- **Transition Event 1 (Brightening):** 150 pixels transition from intensity 8 to 13:

$$d_1 = 13 - 8 = +5 \implies n_D(+5) = 150. \tag{17}$$

- **Transition Event 2 (Darkening):** 90 pixels transition from intensity 18 to 14:

$$d_2 = 14 - 18 = -4 \implies n_D(-4) = 90. \tag{18}$$

- **Stationary Field (Unchanged):** All remaining pixels in Ω maintain identical values across acquisitions:

$$d_3 = 0 \implies n_D(0) = |\Omega| - (150 + 90) = 2000 - 240 = 1760. \tag{19}$$

1.3.2 Histogram of D and Final Classification

The complete non-zero histogram bins of the signed difference image D are listed in Table **??**.

Table 3. Non-Zero Histogram Bins of the Signed Difference Image D.

Difference Intensity (d)	**Pixel Count ($n_D(d)$)**	**Dynamic Classification**
-4	90	Darker ($D < 0$)
0	1760	Unchanged ($D = 0$)
$+5$	150	Brighter ($D > 0$)

Based on the derived difference operator response:

- **Number of pixels that became brighter ($D > 0$): 150** pixels.
- **Number of pixels that became darker ($D < 0$): 90** pixels.
- **Number of pixels that remained unchanged ($D = 0$): 1760** pixels.

Total Population Check: $150 + 90 + 1760 = 2000 = |\Omega|$.

2 Problem 2: Pixel Adjacency and Distance (15 pts)

2.1 Matrix Domain and Coordinate Specification

Consider the 6×6 discrete sub-image $\mathbf{M} \in \mathcal{M}_{6\times 6}(\mathbb{Z})$ indexed by standard 1-based matrix coordinates $(\text{row}, \text{column})$:

$$\mathbf{M} = \begin{bmatrix} 1 & 2 & 0 & 3 & 2 & 1 \\ 0 & 2 & 3 & 0 & 1 & 1 \\ 1 & 0 & 2 & 1 & 2 & \mathbf{0} \\ 2 & 2 & 3 & 0 & 1 & 0 \\ 3 & 1 & 2 & 2 & 2 & 2 \\ 1 & 0 & 1 & 3 & \mathbf{2} & 2 \end{bmatrix}. \tag{20}$$

The designated terminal points are:

$$p = (3, 6) \implies \mathbf{M}(3, 6) = 0, \tag{21}$$
$$q = (6, 5) \implies \mathbf{M}(6, 5) = 2. \tag{22}$$

2.2 (a) Topological Path Search for $V = \{0, 1, 2\}$ (6 pts)

A digital path between p and q is a sequence of distinct spatial coordinates $\langle (x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n) \rangle$ such that $(x_0, y_0) = p$, $(x_n, y_n) = q$, every adjacent pair is connected under the specified adjacency criterion, and all pixel values along the path satisfy $\mathbf{M}(x_i, y_i) \in V$. The path length equals the edge count n.

2.2.1 1. Shortest 4-Path

4-adjacency restricts transitions to the orthogonal neighborhood N_4. Because the theoretical lower bound on 4-path length is dictated by the ℓ_1 Manhattan metric:

$$n \geq \|\Delta \mathbf{x}\|_1 = |3 - 6| + |6 - 5| = 3 + 1 = 4, \tag{23}$$

we seek a monotonic sequence of length 4. A valid shortest 4-path sequence is:

$$\mathbf{P}_4 = \langle (3, 6),\ (4, 6),\ (5, 6),\ (5, 5),\ (6, 5) \rangle. \tag{24}$$

Verification of Intensity Values:

$$\mathbf{M}(3, 6) = 0 \in V, \quad \mathbf{M}(4, 6) = 0 \in V, \quad \mathbf{M}(5, 6) = 2 \in V, \quad \mathbf{M}(5, 5) = 2 \in V, \quad \mathbf{M}(6, 5) = 2 \in V. \tag{25}$$

Shortest 4-path length: 4.

2.2.2 2. Shortest 8-Path

8-adjacency permits transitions to N_8. The lower bound is governed by the Chebyshev metric:

$$n \geq \|\Delta \mathbf{x}\|_\infty = \max(|3 - 6|, |6 - 5|) = \max(3, 1) = 3. \tag{26}$$

Taking diagonal steps to minimize transitions, a shortest 8-path sequence of length 3 is:

$$\mathbf{P}_8 = \langle (3, 6),\ (4, 5),\ (5, 5),\ (6, 5) \rangle. \tag{27}$$

Verification of Intensity Values:

$$\mathbf{M}(3, 6) = 0 \in V, \quad \mathbf{M}(4, 5) = 1 \in V, \quad \mathbf{M}(5, 5) = 2 \in V, \quad \mathbf{M}(6, 5) = 2 \in V. \tag{28}$$

Shortest 8-path length: 3.

2.2.3 3. Shortest m-Path (Mixed Adjacency)

By formal definition, two pixels u and v with values in V are m-adjacent if:

(i) $v \in N_4(u)$, or

(ii) $v \in N_D(u)$ and the intersection set $N_4(u) \cap N_4(v)$ contains **no pixels** whose values belong to V (i.e., $N_4(u) \cap N_4(v) \cap V = \emptyset$).

We evaluate the candidate diagonal transition $(3,6) \to (4,5)$ from the length-3 8-path:

$$N_4(3,6) \cap N_4(4,5) = \{(3,5),\ (4,6)\}\,. \tag{29}$$

Inspecting the lattice values: $\mathbf{M}(3,5) = 2 \in V$ and $\mathbf{M}(4,6) = 0 \in V$. Because both intermediate 4-neighbors belong to V, condition (ii) is violated; hence, $(3,6)$ and $(4,5)$ are **not** m-adjacent. Similarly, for the diagonal step $(4,6) \to (5,5)$, $N_4(4,6) \cap N_4(5,5) = \{(4,5),(5,6)\}$ where $\mathbf{M}(4,5) = 1 \in V$ and $\mathbf{M}(5,6) = 2 \in V$, which also violates condition (ii).

Because any 4-neighbor transition unconditionally satisfies condition (i), every 4-path is strictly an m-path. Since all length-3 8-paths require diagonal jumps that fail condition (ii), the shortest m-path must have length 4. A valid shortest m-path is:

$$\mathbf{P}_m = \langle (3,6),\ (4,6),\ (5,6),\ (5,5),\ (6,5) \rangle. \tag{30}$$

Shortest m-path length: 4.

2.3 (b) Path Reachability Analysis for $V = \{2,3\}$ (6 pts)

We inspect the terminal nodes under intensity set $V = \{2,3\}$:

- The start pixel $p = (3,6)$ has intensity $\mathbf{M}(3,6) = 0$.
- Set membership check: $0 \notin V = \{2,3\}$.

Mathematical Verdict: By definition, every pixel in an admissible digital path must have its intensity in V. Because the start pixel p violates this condition ($\mathbf{M}(p) \notin V$), **no path exists** under 4-, 8-, or m-adjacency.

- **4-path: Does not exist.**
- **8-path: Does not exist.**
- **m-path: Does not exist.**

Justification: The starting coordinate $p = (3,6)$ violates the prerequisite intensity criterion $\mathbf{M}(p) \in V$.

2.4 (c) Geometric Distance Metrics Comparison (3 pts)

Let $r = (1,1)$, $a = (1,6)$, and $b = (4,4)$. Pixel intensity values are ignored.

2.4.1 Distance Computations

1. **City Block Distance (D_4, ℓ_1 norm):**

$$D_4(r,a) = |1-1| + |1-6| = 0 + 5 = \mathbf{5} \tag{31}$$
$$D_4(r,b) = |1-4| + |1-4| = 3 + 3 = \mathbf{6}. \tag{32}$$

2. **Chessboard Distance (D_8, ℓ_∞ norm):**

$$D_8(r,a) = \max(|1-1|, |1-6|) = \max(0,5) = \mathbf{5} \tag{33}$$
$$D_8(r,b) = \max(|1-4|, |1-4|) = \max(3,3) = \mathbf{3}. \tag{34}$$

2.4.2 Metric Ordering and Mathematical Explanation

- Under D_4: $D_4(r,a) = 5 < D_4(r,b) = 6 \implies$ **Pixel a is closer to r.**
- Under D_8: $D_8(r,b) = 3 < D_8(r,a) = 5 \implies$ **Pixel b is closer to r.**

Mathematical Explanation for Divergence:
D_4 corresponds to the ℓ_1 norm ($\|\Delta\mathbf{x}\|_1 = |\Delta x| + |\Delta y|$), whose equidistant contours form diamonds tilted at 45°. Diagonal displacement is heavily penalized as the sum of orthogonal components ($\Delta x + \Delta y$). For pixel b, $\Delta\mathbf{x} = (3,3)$ incurs a full distance of $3+3=6$.

D_8 corresponds to the ℓ_∞ norm ($\|\Delta\mathbf{x}\|_\infty = \max(|\Delta x|, |\Delta y|)$), whose equidistant contours are axis-aligned squares. A diagonal step costs exactly the same as a purely horizontal or vertical step ($\max(1,1) = 1$). For pixel b, its simultaneous displacement in both directions is compressed into $\max(3,3) = 3$.

In contrast, displacement to a is purely horizontal ($\Delta\mathbf{x} = (0,5)$). Along coordinate axes, the ℓ_1 and ℓ_∞ norms coincide ($\|\Delta\mathbf{x}\|_1 = \|\Delta\mathbf{x}\|_\infty = 5$). Hence, D_8 significantly privileges diagonal trajectories over axial ones, reversing the spatial proximity ordering.

3 Problem 3: Spatial Filtering and Histogram Information (25 pts)

3.1 Input Specifications & Filter Kernel

Consider two binary images $A, B \in \mathcal{M}_{10\times10}(\{0,1\})$. Both images contain exactly 50 zero-valued pixels and 50 one-valued pixels ($|\Omega| = 100$):

- **Image A (Low-frequency step field):** Columns 1–5 are 0; Columns 6–10 are 1. Formally:

$$A(x,y) = \begin{cases} 0, & 1 \le y \le 5 \\ 1, & 6 \le y \le 10 \end{cases}, \quad \forall x \in \{1,\dots,10\}. \tag{35}$$

- **Image B (High-frequency alternating field):** Columns alternate between 0 and 1 identically across every row:

$$B(x,y) = \begin{cases} 0, & y \in \{1,3,5,7,9\} \\ 1, & y \in \{2,4,6,8,10\} \end{cases}, \quad \forall x \in \{1,\dots,10\}. \tag{36}$$

The spatial filter is a 3×3 box averaging filter $h \in \mathcal{M}_{3\times3}(\mathbb{R})$ applied with stride $s = 1$:

$$h = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \tag{37}$$

3.2 (a) Output Dimensions and Padding Requirements (5 pts)

Let the input dimensions be $M \times N = 10 \times 10$, and kernel dimensions be $m \times n = 3 \times 3$.

1. **Valid Output Dimensions (Without Padding):** Filtering without padding restricts evaluation exclusively to spatial coordinates where the entire 3×3 kernel fully overlaps with the image support:

$$M_{\text{out}} = M - m + 1 = 10 - 3 + 1 = \mathbf{8} \tag{38}$$
$$N_{\text{out}} = N - n + 1 = 10 - 3 + 1 = \mathbf{8}. \tag{39}$$

   Thus, the output size is $\mathbf{8} \times \mathbf{8}$ (totaling 64 pixels).

2. **Padding for Dimension Invariance (10×10 Output):** To maintain 10×10, the required symmetric zero-padding is:

$$P_{\text{top}} = 1, \quad P_{\text{bottom}} = 1, \quad P_{\text{left}} = 1, \quad P_{\text{right}} = 1. \tag{40}$$

   Specifically, **1 row on top, 1 row on bottom, 1 column on the left, and 1 column on the right**.

3.3 (b) Histogram Isomorphism Under Valid Filtering (5 pts)

Answer: NO.
Formal Justification:
Spatial averaging computes the local empirical mean over each 3×3 support window:

$$g(x, y) = \frac{1}{9} \sum_{s=-1}^{1} \sum_{t=-1}^{1} f(x + s, y + t). \tag{41}$$

- **Filtered Image A:** Image A consists of two large homogeneous contiguous regions separated by a single vertical boundary between columns 5 and 6. For any 3×3 window residing completely within columns 1–5, all 9 covered pixels are 0, producing an output intensity of $0/9 = 0$. Similarly, any window within columns 6–10 produces an intensity of $9/9 = 1$. The only intermediate values (3/9 and 6/9) occur at the single transition boundary. Hence, the filtered output of A retains a large fraction of pure 0 and pure 1 values.
- **Filtered Image B:** Image B oscillates at the spatial Nyquist frequency along the horizontal axis $(0, 1, 0, 1, \dots)$. Any 3×3 neighborhood spans three consecutive alternating columns, which unconditionally contain either $\{0, 1, 0\}$ (sum $= 1 \times 3 = 3$) or $\{1, 0, 1\}$ (sum $= 2 \times 3 = 6$). Consequently, the local window sum is strictly either 3 or 6, mapping every output pixel to either 3/9 or 6/9. Filtered B contains **zero pixels** with intensity 0 or 1.

Because the supports of their resulting intensity distributions are entirely different ($\text{supp}(n_A) = \{0, 3/9, 6/9, 1\}$ versus $\text{supp}(n_B) = \{3/9, 6/9\}$), their histograms cannot be identical.

3.4 (c) Exact Non-Zero Histogram Calculation for 8×8 Output (10 pts)

Because both images are invariant along rows, every row of the 8×8 output yields identical column profiles.

3.4.1 1. Filtered Image A

Across the 8 output columns (input column spans $\{j, j + 1, j + 2\}$):

- Cols 1–3: Covered inputs are all $\{0, 0, 0\} \implies$ Window sum $0 \implies$ Intensity **0**. Count: $3 \times 8 = \mathbf{24}$.
- Col 4: Covered inputs are $\{0, 0, 1\} \implies$ Window sum $3 \implies$ Intensity **3/9**. Count: $1 \times 8 = \mathbf{8}$.
- Col 5: Covered inputs are $\{0, 1, 1\} \implies$ Window sum $6 \implies$ Intensity **6/9**. Count: $1 \times 8 = \mathbf{8}$.
- Cols 6–8: Covered inputs are all $\{1, 1, 1\} \implies$ Window sum $9 \implies$ Intensity **1**. Count: $3 \times 8 = \mathbf{24}$.

3.4.2 2. Filtered Image B

Across the 8 output columns:

- Odd cols (1, 3, 5, 7): Covered inputs are $\{0, 1, 0\} \implies$ Window sum 3 $\implies$ Intensity $\mathbf{3/9}$. Count: $4 \times 8 = \mathbf{32}$.
- Even cols (2, 4, 6, 8): Covered inputs are $\{1, 0, 1\} \implies$ Window sum 6 $\implies$ Intensity $\mathbf{6/9}$. Count: $4 \times 8 = \mathbf{32}$.

Table 4. Non-Zero Histogram Bins for 8×8 Valid Filtered Outputs.

Intensity Level	**Filtered Image A**		**Filtered Image B**	
	Pixel Count	**Percentage**	**Pixel Count**	**Percentage**
0 (0/9)	**24**	37.5%	**0**	0.0%
3/9 (1/3)	**8**	12.5%	**32**	50.0%
6/9 (2/3)	**8**	12.5%	**32**	50.0%
1 (9/9)	**24**	37.5%	**0**	0.0%
Total	**64**	100.0%	**64**	100.0%

3.5 (d) Interpretation (5 pts)

An image histogram is a **first-order marginal statistic** that records only the global probability distribution of pixel intensities while completely discarding all spatial arrangement, spatial geometry, and higher-order joint spatial correlations. Conversely, spatial filtering is fundamentally an **operator on local spatial configurations**; thus, images with identical global histograms but distinct spatial topologies inevitably yield drastically different histograms after filtering.

4 Problem 4: Recovering an Affine Transformation (15 pts)

4.1 Affine Transformation Model and Point Correspondences

In 2D Euclidean space represented in homogeneous coordinates, a general affine transformation $\mathcal{T}_{\mathbf{A}} : \mathbb{R}^2 \to \mathbb{R}^2$ is parameterized by six independent scalar parameters:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{A} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{42}$$

Expanding into standard Cartesian coordinate expressions yields the coupled linear system:

$$\begin{cases} x' = a_{11}x + a_{12}y + a_{13} \\ y' = a_{21}x + a_{22}y + a_{23}. \end{cases} \tag{43}$$

We are provided three point correspondences $(x_i, y_i) \to (x'_i, y'_i)$ for $i \in \{1, 2, 3\}$:

$$P_1 : (1, 1) \longrightarrow (4, 3) \tag{44}$$
$$P_2 : (3, 1) \longrightarrow (6, 1) \tag{45}$$
$$P_3 : (1, 3) \longrightarrow (6, 5). \tag{46}$$

4.2 (a) Transformation Recovery via Linear System Inversion (8 pts)

Because the mappings for x' and y' share the identical spatial basis functions $(x, y, 1)$, the six scalar equations decouple into two independent 3×3 linear systems sharing the same design matrix $\mathbf{X} \in \mathcal{M}_{3\times3}(\mathbb{R})$:

$$\mathbf{X}\mathbf{a}_x = \mathbf{b}_x, \quad \mathbf{X}\mathbf{a}_y = \mathbf{b}_y, \tag{47}$$

where:

$$\mathbf{X} = \begin{bmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 3 & 1 & 1 \\ 1 & 3 & 1 \end{bmatrix}, \quad \mathbf{a}_x = \begin{bmatrix} a_{11} \\ a_{12} \\ a_{13} \end{bmatrix}, \quad \mathbf{a}_y = \begin{bmatrix} a_{21} \\ a_{22} \\ a_{23} \end{bmatrix}, \quad \mathbf{b}_x = \begin{bmatrix} 4 \\ 6 \\ 6 \end{bmatrix}, \quad \mathbf{b}_y = \begin{bmatrix} 3 \\ 1 \\ 5 \end{bmatrix}. \tag{48}$$

4.2.1 1. Invertibility of the Design Matrix

We compute the determinant of $\mathbf{X}$ by cofactor expansion along the first row:

$$\det(\mathbf{X}) = 1 \cdot \begin{vmatrix} 1 & 1 \\ 3 & 1 \end{vmatrix} - 1 \cdot \begin{vmatrix} 3 & 1 \\ 1 & 1 \end{vmatrix} + 1 \cdot \begin{vmatrix} 3 & 1 \\ 1 & 3 \end{vmatrix} \tag{49}$$

$$= 1(1-3) - 1(3-1) + 1(9-1) \tag{50}$$

$$= -2 - 2 + 8 = \mathbf{4} \neq 0. \tag{51}$$

Because $\det(\mathbf{X}) = 4 \neq 0$, the design matrix $\mathbf{X}$ is strictly non-singular ($\text{rank}(\mathbf{X}) = 3$), guaranteeing a unique parameter solution.

4.2.2 2. Gaussian Elimination for x-Axis Parameters

The linear system for the first row parameters is:

$$a_{11} + a_{12} + a_{13} = 4 \tag{52}$$

$$3a_{11} + a_{12} + a_{13} = 6 \tag{53}$$

$$a_{11} + 3a_{12} + a_{13} = 6 \tag{54}$$

Subtracting Equation (**??**) from Equation (**??**):

$$(3a_{11} + a_{12} + a_{13}) - (a_{11} + a_{12} + a_{13}) = 6 - 4 \implies 2a_{11} = 2 \implies \mathbf{a_{11} = 1}. \tag{55}$$

Subtracting Equation (**??**) from Equation (**??**):

$$(a_{11} + 3a_{12} + a_{13}) - (a_{11} + a_{12} + a_{13}) = 6 - 4 \implies 2a_{12} = 2 \implies \mathbf{a_{12} = 1}. \tag{56}$$

Substituting $a_{11} = 1$ and $a_{12} = 1$ back into Equation (**??**):

$$1(1) + 1(1) + a_{13} = 4 \implies 2 + a_{13} = 4 \implies \mathbf{a_{13} = 2}. \tag{57}$$

4.2.3 3. Gaussian Elimination for y-Axis Parameters

The linear system for the second row parameters is:

$$a_{21} + a_{22} + a_{23} = 3 \tag{58}$$

$$3a_{21} + a_{22} + a_{23} = 1 \tag{59}$$

$$a_{21} + 3a_{22} + a_{23} = 5 \tag{60}$$

Subtracting Equation (**??**) from Equation (**??**):

$$(3a_{21} + a_{22} + a_{23}) - (a_{21} + a_{22} + a_{23}) = 1 - 3 \implies 2a_{21} = -2 \implies \mathbf{a_{21} = -1}. \tag{61}$$

Subtracting Equation (**??**) from Equation (**??**):

$$(a_{21} + 3a_{22} + a_{23}) - (a_{21} + a_{22} + a_{23}) = 5 - 3 \implies 2a_{22} = 2 \implies \mathbf{a_{22} = 1}. \tag{62}$$

Substituting $a_{21} = -1$ and $a_{22} = 1$ back into Equation (**??**):

$$(-1) + 1 + a_{23} = 3 \implies \mathbf{a_{23} = 3}. \tag{63}$$

4.2.4 4. Complete Affine Transformation Matrix

Assembling the recovered coefficients yields the complete homogeneous affine transformation matrix:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 \\ -1 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix}. \tag{64}$$

4.3 (b) Forward Coordinate Mapping of New Point (4 pts)

We apply the recovered affine transformation $\mathbf{A}$ to the test coordinate $(x, y) = (2, 4)$:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 \\ -1 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ 1 \end{bmatrix}. \tag{65}$$

Performing the matrix-vector multiplication:

$$x' = 1(2) + 1(4) + 2(1) = 2 + 4 + 2 = \mathbf{8} \tag{66}$$

$$y' = -1(2) + 1(4) + 3(1) = -2 + 4 + 3 = \mathbf{5}. \tag{67}$$

Thus, the transformed coordinates of $(2, 4)$ are $(\mathbf{x}', \mathbf{y}') = (\mathbf{8}, \mathbf{5})$.

4.4 (c) Degenerate Configuration Analysis (3 pts)

Linear Algebraic Proof:
Suppose the three source points $P_1(x_1, y_1)$, $P_2(x_2, y_2)$, and $P_3(x_3, y_3)$ lie on the same straight line $\mathcal{L} \subset \mathbb{R}^2$. By definition of a 2D line, there exist non-trivial constants $(\alpha, \beta, \gamma) \in \mathbb{R}^3$ with $(\alpha, \beta) \neq (0, 0)$ such that:

$$\alpha x_i + \beta y_i + \gamma = 0, \quad \forall i \in \{1, 2, 3\}. \tag{68}$$

Expressing this condition in terms of the column vectors $\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$ of the design matrix $\mathbf{X}$:

$$\alpha \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \beta \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} + \gamma \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \alpha\mathbf{c}_1 + \beta\mathbf{c}_2 + \gamma\mathbf{c}_3 = \mathbf{0}. \tag{69}$$

Because a non-trivial linear combination of the columns equals the zero vector, the column vectors of $\mathbf{X}$ are linearly dependent. Consequently:

$$\det(\mathbf{X}) = 0 \iff \operatorname{rank}(\mathbf{X}) \leq 2 < 3. \tag{70}$$

Geometric & Degrees of Freedom Interpretation:
An affine transformation possesses 6 degrees of freedom, consisting of 2D translation, rotation, scaling, and shear along two linearly independent spatial directions. Three collinear points span only a 1D affine subspace (a line); hence, they constrain transformation behaviors (scaling and translation) strictly along the direction of that line. Any deformation, shear, or scaling orthogonal to the line lies within the null space null($\mathbf{X}$) and remains entirely unconstrained. Consequently, the linear systems $\mathbf{X}\mathbf{a}_x = \mathbf{b}_x$ and $\mathbf{X}\mathbf{a}_y = \mathbf{b}_y$ are rank-deficient, possessing infinitely many valid parameter solutions. Three collinear points are therefore fundamentally insufficient to uniquely resolve an affine transformation in $\mathbb{R}^2$.

5 Problem 5: Histogram Equalization (15 pts)

5.1 Discrete Mathematical Framework & Quantization Baseline

Consider a 3-bit discrete grayscale image ($L = 2^3 = 8$) defined on a spatial grid of size $M \times N = 30 \times 20$. The total pixel population is $|\Omega| = 600$. The discrete intensity levels are indexed by $r_k \in \{0, 1, 2, \ldots, L-1\} = \{0, 1, \ldots, 7\}$. The discrete probability of occurrence of intensity level r_k is given by:

$$p_r(r_k) = \frac{n_k}{|\Omega|} = \frac{n_k}{600}, \quad k = 0, 1, \ldots, 7, \tag{71}$$

where n_k is the absolute pixel count of intensity r_k. The discrete cumulative distribution function (CDF) is formalized as:

$$\mathrm{CDF}(r_k) = \sum_{j=0}^{k} p_r(r_j) = \frac{1}{|\Omega|} \sum_{j=0}^{k} n_j. \tag{72}$$

The discrete histogram equalization transformation $T : \mathbb{Z}_L \to \mathbb{Z}_L$ maps each input level r_k to an output level s_k via:

$$s_k = T(r_k) = \mathrm{round}\left[(L-1) \sum_{j=0}^{k} p_r(r_j) \right] = \mathrm{round}\left[7 \cdot \mathrm{CDF}(r_k) \right]. \tag{73}$$

5.2 (a) Step-by-Step Equalization Mapping Derivation (8 pts)

We evaluate the discrete mapping operator $\mathrm{round}[7 \cdot \mathrm{CDF}(r_k)]$ sequentially across all intensity levels $k \in \{0, \ldots, 7\}$:

1. **Level $r_0 = 0$ ($n_0 = 30$):**

$$p_r(0) = \frac{30}{600} = 0.0500 \tag{74}$$
$$\mathrm{CDF}(0) = 0.0500 \tag{75}$$
$$7 \cdot \mathrm{CDF}(0) = 7 \times 0.0500 = 0.3500 \implies s_0 = \mathrm{round}(0.3500) = \mathbf{0}. \tag{76}$$

2. **Level $r_1 = 1$ ($n_1 = 60$):**

$$p_r(1) = \frac{60}{600} = 0.1000 \tag{77}$$
$$\mathrm{CDF}(1) = 0.0500 + 0.1000 = 0.1500 \tag{78}$$
$$7 \cdot \mathrm{CDF}(1) = 7 \times 0.1500 = 1.0500 \implies s_1 = \mathrm{round}(1.0500) = \mathbf{1}. \tag{79}$$

3. **Level $r_2 = 2$ ($n_2 = 90$):**

$$p_r(2) = \frac{90}{600} = 0.1500 \tag{80}$$
$$\mathrm{CDF}(2) = 0.1500 + 0.1500 = 0.3000 \tag{81}$$
$$7 \cdot \mathrm{CDF}(2) = 7 \times 0.3000 = 2.1000 \implies s_2 = \mathrm{round}(2.1000) = \mathbf{2}. \tag{82}$$

4. **Level $r_3 = 3$ ($n_3 = 150$):**

$$p_r(3) = \frac{150}{600} = 0.2500 \tag{83}$$
$$\mathrm{CDF}(3) = 0.3000 + 0.2500 = 0.5500 \tag{84}$$
$$7 \cdot \mathrm{CDF}(3) = 7 \times 0.5500 = 3.8500 \implies s_3 = \mathrm{round}(3.8500) = \mathbf{4}. \tag{85}$$

5. **Level $r_4 = 4$ ($n_4 = 120$):**

$$p_r(4) = \frac{120}{600} = 0.2000 \tag{86}$$
$$\text{CDF}(4) = 0.5500 + 0.2000 = 0.7500 \tag{87}$$
$$7 \cdot \text{CDF}(4) = 7 \times 0.7500 = 5.2500 \implies s_4 = \text{round}(5.2500) = \mathbf{5}. \tag{88}$$

6. **Level $r_5 = 5$ ($n_5 = 90$):**

$$p_r(5) = \frac{90}{600} = 0.1500 \tag{89}$$
$$\text{CDF}(5) = 0.7500 + 0.1500 = 0.9000 \tag{90}$$
$$7 \cdot \text{CDF}(5) = 7 \times 0.9000 = 6.3000 \implies s_5 = \text{round}(6.3000) = \mathbf{6}. \tag{91}$$

7. **Level $r_6 = 6$ ($n_6 = 40$):**

$$p_r(6) = \frac{40}{600} \approx 0.0667 \tag{92}$$
$$\text{CDF}(6) = \frac{540 + 40}{600} = \frac{580}{600} \approx 0.9667 \tag{93}$$
$$7 \cdot \text{CDF}(6) = 7 \times \frac{580}{600} = \frac{4060}{600} \approx 6.7667 \implies s_6 = \text{round}(6.7667) = \mathbf{7}. \tag{94}$$

8. **Level $r_7 = 7$ ($n_7 = 20$):**

$$p_r(7) = \frac{20}{600} \approx 0.0333 \tag{95}$$
$$\text{CDF}(7) = \frac{580 + 20}{600} = 1.0000 \tag{96}$$
$$7 \cdot \text{CDF}(7) = 7 \times 1.0000 = 7.0000 \implies s_7 = \text{round}(7.0000) = \mathbf{7}. \tag{97}$$

The full analytical pipeline for discrete histogram equalization is summarized in Table **??**.

Table 5. Discrete Histogram Equalization Mapping Pipeline ($M \times N = 600, L = 8$).

r_k	n_k	$p_r(r_k) = n_k/600$	$\text{CDF}(r_k)$	$(L-1) \cdot \text{CDF}$	**Exact Real Value**	$s_k = \text{round}(\cdot)$
0	30	0.0500	0.0500	7×0.0500	0.3500	**0**
1	60	0.1000	0.1500	7×0.1500	1.0500	**1**
2	90	0.1500	0.3000	7×0.3000	2.1000	**2**
3	150	0.2500	0.5500	7×0.5500	3.8500	**4**
4	120	0.2000	0.7500	7×0.7500	5.2500	**5**
5	90	0.1500	0.9000	7×0.9000	6.3000	**6**
6	40	0.0667	0.9667	7×0.9667	6.7667	**7**
7	20	0.0333	1.0000	7×1.0000	7.0000	**7**

5.3 (b) Equalized Histogram Reconstruction (4 pts)

Let $n_s(s)$ denote the pixel count of intensity s in the transformed image, formed by aggregating all pre-image bins under T:

$$n_s(s) = \sum_{k:s_k=s} n_k. \tag{98}$$

Evaluating across all output levels $s \in \{0, 1, \ldots, 7\}$:

- $s = 0 \longleftarrow r_0 \implies n_s(0) = n_0 = \mathbf{30}$.
- $s = 1 \longleftarrow r_1 \implies n_s(1) = n_1 = \mathbf{60}$.
- $s = 2 \longleftarrow r_2 \implies n_s(2) = n_2 = \mathbf{90}$.
- $s = 3 \longleftarrow \emptyset \implies n_s(3) = \mathbf{0}$ (*Unmapped empty bin*).
- $s = 4 \longleftarrow r_3 \implies n_s(4) = n_3 = \mathbf{150}$.
- $s = 5 \longleftarrow r_4 \implies n_s(5) = n_4 = \mathbf{120}$.
- $s = 6 \longleftarrow r_5 \implies n_s(6) = n_5 = \mathbf{90}$.
- $s = 7 \longleftarrow \{r_6, r_7\} \implies n_s(7) = n_6 + n_7 = 40 + 20 = \mathbf{60}$.

Table 6. Complete Reconstructed Histogram of the Equalized Image.

Equalized Intensity (s)	**Pixel Count** ($n_s(s)$)	**Probability** ($p_s(s) = n_s/600$)	**Pre-image Source L**
0	30	$30/600 = 0.0500$	$\{r_0\}$
1	60	$60/600 = 0.1000$	$\{r_1\}$
2	90	$90/600 = 0.1500$	$\{r_2\}$
3	0	$0/600 = 0.0000$	$\emptyset$ (*Empty*)
4	150	$150/600 = 0.2500$	$\{r_3\}$
5	120	$120/600 = 0.2000$	$\{r_4\}$
6	90	$90/600 = 0.1500$	$\{r_5\}$
7	60	$60/600 = 0.1000$	$\{r_6, r_7\}$ (*Merged*)
Total	**600**	**1.0000**	$\lvert\Omega\rvert = M \times N$

Measure Conservation Verification:

$$\sum_{s=0}^{7} n_s(s) = 30 + 60 + 90 + 0 + 150 + 120 + 90 + 60 = 600 = |\Omega|. \quad \text{(Q.E.D.)} \tag{99}$$

5.4 (c) Interpretation and Degeneracy Analysis (3 pts)

1. Uniformity Verdict:
The resulting histogram is **not perfectly uniform** (a perfectly uniform distribution across 8 bins would require exactly $600/8 = 75$ pixels per intensity level).
2. Mathematical Mechanism of Non-Uniformity:
In continuous probability theory, if a continuous random variable $r \sim p_r(r)$ is transformed by its continuous cumulative distribution function $s = T(r) = (L-1)\int_0^r p_r(w)dw$, the transformed variable s is rigorously guaranteed to follow a uniform probability density $p_s(s) = \frac{1}{L-1}$.

However, for a digital image on a discrete lattice, the pixel intensity range is a finite discrete integer ring $\mathbb{Z}_L$, and the histogram is a sum of discrete Dirac impulses. Consequently, the cumulative distribution function is a **monotonically non-decreasing step function**. The transformation incorporates a non-linear integer quantization rounding operator round$(\cdot)$:

- **Many-to-One Degeneracy (Bin Merging):** Distinct adjacent input levels with modest probability increments can round to the identical integer level. Specifically, **input levels $r_6 = 6$ and $r_7 = 7$ are both mapped to output level $s = 7$**:

$$T(6) = 7, \quad T(7) = 7. \tag{100}$$

- **Sparsity / Empty Bins (Gaps):** Whenever a step in the cumulative sum $(L-1)\Delta\,\mathrm{CDF} = (L-1)p_r(r_k)$ exceeds 1.0, the rounding operator skips one or more discrete integer levels. Here, between r_2 and r_3, the scaled value jumps from 2.10 to 3.85, jumping across the rounding basin of 3. Thus, **output level $s = 3$ receives zero pixels** ($n_s(3) = 0$).

Because pixels at the same input level cannot be subdivided without spatial context, discrete histogram equalization can only shift and merge entire bins, necessarily yielding a non-uniform, discrete histogram with irregular bin heights and empty intermediate levels.

6 Programming Problem 6: Laplacian Filtering and Bit-Plane Watermarking (20 pts)

6.1 (a) Laplacian Edge Extraction and Highpass Sharpening (12 pts)

6.1.1 1. Second-Order Isotropic Derivative Operators

To highlight intensity transitions in all directions while strictly avoiding built-in filter functions, we implement spatial discrete convolution using the standard second-order isotropic Laplacian operators with a negative center coefficient (Lecture 4, p.70):

- **4-Neighborhood Laplacian Operator (K_4):**

$$K_4 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \nabla^2_{K_4} f(x,y) = f(x{+}1,y)+f(x{-}1,y)+f(x,y{+}1)+f(x,y{-}1)-4f(x,y). \tag{101}$$

- **8-Neighborhood Isotropic Laplacian Operator (K_8):** Incorporates diagonal finite differences to achieve complete rotational isotropy:

$$K_8 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \tag{102}$$

6.1.2 2. Boundary Handling and Custom Native Convolution

To prevent dimension decay while respecting boundary conditions, input double-precision matrix $f \in \mathbb{R}^{M\times N}$ is symmetrically zero-padded with border width $p = 1$, constructing $f_{\text{pad}} \in \mathbb{R}^{(M+2)\times(N+2)}$. For each pixel (i, j), discrete spatial convolution is evaluated directly as the Frobenius inner product:

$$\text{lap}(i,j) = \sum_{s=-1}^{1}\sum_{t=-1}^{1} K(s+2,t+2)\cdot f_{\text{pad}}(i+s+1, j+t+1). \tag{103}$$

Because both K_4 and K_8 are centrosymmetric ($K(-s,-t) = K(s,t)$), spatial correlation is algebraically equivalent to spatial convolution.

6.1.3 3. Highpass Sharpening Synthesis and Clamping

The sharpened image $g(x, y)$ is synthesized by adding the edge map back to the original image:

$$g(x,y) = f(x,y) + c\cdot\nabla^2 f(x,y). \tag{104}$$

Because both K_4 and K_8 have negative center coefficients (-4 and -8), the scaling constant is set to $c = -1$, yielding:

$$g_4 = f - \text{lap4}, \quad g_8 = f - \text{lap8}\,. \tag{105}$$

To prevent integer underflow and overflow in the reconstructed image, we enforce strict dynamic range saturation clamping before casting to `uint8`:

$$\text{sharp4} = \text{uint8}\left(\min\left(\max(g_4, 0), 255\right)\right), \quad \text{sharp8} = \text{uint8}\left(\min\left(\max(g_8, 0), 255\right)\right). \tag{106}$$

6.2 (b) Bit-Plane Slicing and Watermark Recovery (8 pts)

Any 8-bit grayscale pixel intensity $f(x, y) \in [0, 255]$ can be represented as an orthogonal sum of binary basis functions:

$$f(x, y) = \sum_{k=1}^{8} a_k(x, y) \cdot 2^{k-1}, \quad a_k(x, y) \in \{0, 1\}. \tag{107}$$

Using the `bitget` operator, each bit plane $k \in \{1, \ldots, 8\}$ is extracted and polarized to the display dynamic range $\{0, 255\}$ via $\text{planes}(:, :, k) = \text{uint8}(\text{bitget}(img, k) \times 255)$.

Official Written Answer Statement:

The hidden watermark is embedded in Bit Plane 1 (Least Significant Bit, LSB).

Steganographic Rationale:
Higher bit planes ($k = 5, \ldots, 8$) encode the dominant structural topology and macroscopic contrast of the cameraman scene. In contrast, lower bit planes exhibit high-entropy pseudo-random noise patterns. In this dataset, Bit Plane 1 ($k = 1$) reveals a zero-noise, deterministic binary watermark encoding the official emblem of The Hong Kong University of Science and Technology (HKUST). Embedding within the LSB introduces a maximum spatial perturbation of only ± 1 gray level, ensuring that the watermark remains imperceptible to human visual inspection.

6.3 Visual Inspection of Experimental Results

The resulting highpass sharpened images and the extracted binary watermark are illustrated in Figure **??**.



(a) Sharpened image using K_4 ($c = -1$). **(b)** Sharpened image using K_8 ($c = -1$). **(c)** Extracted watermark (Bit Plane 1).

Figure 1. Experimental deliverables for Problem 6: Laplacian highpass sharpening and bit-plane watermark recovery.

Sharpening Comparative Analysis:
Visual comparison between Figures **??** and **??** confirms the theoretical property of isotropic derivatives:

- K_4 **Sharpening:** Enhances edges strictly aligned with the horizontal and vertical coordinate axes (e.g., the silhouette of the coat, the vertical tripod legs, and horizontal roof lines).
- K_8 **Sharpening:** Because diagonal finite differences are incorporated, K_8 provides superior omnidirectional gradient amplification. Oblique features, such as the angled tripod struts and high-frequency grass lawn textures, exhibit enhanced contrast and crispness compared to K_4.

A Appendix A: Complete MATLAB Source Code (`problem6.m`)

Below is the full, unabridged source code of `problem6.m`. The implementation strictly abides by all course constraints: zero built-in filtering routines are utilized, modifications are confined exclusively to the designated `Coding zone` blocks, and execution runs deterministically.

```
%% COMP4421 Assignment 1 - Programming Problem 6 (20 pts)
% Input image: watermarked_cameraman.png (8-bit grayscale)
%
% Rules:
%   * In Question a) you must implement the Laplacian filtering yourself.
%     Do NOT use imfilter, conv2, filter2, fspecial, del2, or any other
%     built-in filtering function.
%   * Only write code at the places marked by "% your code here" inside the
%     "Coding zone" blocks. Modifying any other line of this skeleton gives
%     0 marks for the corresponding sub-question.
%   * In your written answers, state which bit plane (1-8) contains the
    watermark.
%   * Put this script, the three images it generates (sharpened_K4.png,
%     sharpened_K8.png, hidden_watermark.png) and your written answers (PDF)
%     together into one zip file named 20861656.zip.

img = imread("watermarked_cameraman.png");
f   = double(img);                      % work in double to avoid uint8
    overflow
[M, N] = size(img);

% initialized variables for you to answer questions
K4 = zeros(3);                          % a Laplacian kernel
K8 = zeros(3);                          % kernel that includes the diagonal
    terms
lap4 = zeros(M, N);                     % Laplacian response using K4
lap8 = zeros(M, N);                     % Laplacian response using K8
sharp4 = zeros(M, N, "uint8");          % sharpened image using K4
sharp8 = zeros(M, N, "uint8");          % sharpened image using K8
planes = zeros(M, N, 8, "uint8");       % planes(:,:,k) = k-th bit plane (0 or
    255)
watermark = zeros(M, N, "uint8");       % the hidden watermark (0 or 255)

%------------ Coding zone-------------
% Question a) Laplacian edge extraction and sharpening (12 pts)
% (i)   4 pts: Define K4, a Laplacian kernel (see Lecture 4, p.70), filter f
%              with it and assign the response (edge map) to 'lap4'.
% (ii)  4 pts: Define K8, the kernel that includes the diagonal terms, filter
%              f with it and assign the response (edge map) to 'lap8'.
%       For both kernels: use a NEGATIVE centre coefficient, filter f by
%       yourself (no built-in filtering functions), pad the border with
%       zeros (zero padding), and keep the response the same size as f.
% (iii) 2 pts: Sharpen with g = f + c * lap4, clip to [0, 255], convert to
%              uint8, and assign the result to 'sharp4'.
%              Hint: c = -1 if the kernel's centre coefficient is negative,
%                    c = +1 if it is positive.
% (iv)  2 pts: Same as (iii) with lap8; assign the result to 'sharp8'.

% (i) Define K4: 4-neighborhood Laplacian operator with negative center
    coefficient
K4 = [ 0,  1,  0;
       1, -4,  1;
       0,  1,  0];

% (ii) Define K8: 8-neighborhood Laplacian operator (including diagonals) with
    negative center coefficient
K8 = [ 1,  1,  1;
```

```
        1, -8,  1;
        1,  1,  1];

% Border zero-padding (p = 1) to maintain spatial dimensions M x N
f_pad = zeros(M + 2, N + 2);
f_pad(2:M+1, 2:N+1) = f;

% Native 2D spatial convolution implementation (avoiding imfilter, conv2, filter2)
for i = 1:M
    for j = 1:N
        window = f_pad(i:i+2, j:j+2);
        lap4(i, j) = sum(sum(K4 .* window));
        lap8(i, j) = sum(sum(K8 .* window));
    end
end

% (iii) Sharpening synthesis with K4: g = f + c * lap4; c = -1 due to negative center
c = -1;
g4 = f + c * lap4;
g4_clipped = min(max(g4, 0), 255);
sharp4 = uint8(g4_clipped);

% (iv) Sharpening synthesis with K8: g = f + c * lap8; c = -1 due to negative center
g8 = f + c * lap8;
g8_clipped = min(max(g8, 0), 255);
sharp8 = uint8(g8_clipped);

%-----------------END-----------------

%------------ Coding zone-------------
% Question b) Bit-plane slicing (8 pts)
% A watermark is hidden in ONE bit plane of the image. Hint: you may use bitget().
% (i)  4 pts: Slice the image into its 8 bit planes. Store the k-th plane
%             (k = 1 is the least-significant bit) in planes(:,:,k), scaled to 0/255.
% (ii) 4 pts: Find the plane that contains the watermark and assign it to 'watermark'.
%             State which bit plane (1-8) it is in your written answers.

% (i) Deconstruct the 8-bit image into 8 binary bit planes
% Scale the boolean values {0, 1} to full display dynamic range {0, 255}
for k = 1:8
    planes(:, :, k) = uint8(bitget(img, k) * 255);
end

% (ii) Assign the plane containing the watermark to 'watermark'
% Through spatial inspection across all 8 bit planes, Bit Plane 1 (LSB) contains
% the structured binary HKUST watermark emblem.
target_plane = 1;
watermark = planes(:, :, target_plane);

%-----------------END-----------------

% save your results (do not modify)
imwrite(sharp4, "sharpened_K4.png");
imwrite(sharp8, "sharpened_K8.png");
imwrite(watermark, "hidden_watermark.png");      % standalone 8-bit grayscale image
```

```
107
108 % visualize your results
109 figure("Name", "Question a) Laplacian filters");
110 subplot(231); imshow(img);          title("input image");
111 subplot(232); imshow(lap4, []);     title("Laplacian (K4)");
112 subplot(233); imshow(lap8, []);     title("Laplacian (K8)");
113 subplot(235); imshow(sharp4);       title("sharpened (K4)");
114 subplot(236); imshow(sharp8);       title("sharpened (K8)");
115
116 figure("Name", "Question b) Bit planes");
117 for k = 1:8
118     subplot(2, 4, k); imshow(planes(:, :, k)); title("bit plane " + k);
119 end
120
121 figure("Name", "Question b) Hidden watermark");
122 imshow(watermark); title("hidden watermark");
```

Listing 1. Complete and unabridged implementation of `problem6.m`.

B Appendix B: Generative AI Acknowledgment Statement

Compliance with HKUST COMP 4421 Course Policy:
The official syllabus of COMP 4421 explicitly specifies: *"Students are free to use generative AI on their works, and should acknowledge it when used."* In full adherence to this institutional standard and the ethical guidelines of HKUST, this section documents the transparent disclosure of generative AI usage throughout the preparation of Assignment 1.

1. Nature and Scope of AI Consultation

Generative AI tools (specifically large language model interfaces) were consulted strictly in an auxiliary, advisory, and technical editing capacity for:

- **LaTeX Typesetting & Syntax Compilation:** Structuring the multi-section academic template, resolving package conflicts between `geometry`, `fancyhdr`, and `listings`, and formatting publication-grade `booktabs` tables and `subfigure` floating environments.
- **Numerical & Symbolic Verification:** Formulating a standalone Python cross-verification script (`verify_math.py`) to confirm the correctness of hand calculations (e.g., verifying the matrix determinant $\det(\mathbf{X}) = 4$, boundary saturation sums, and BFS shortest-path coordinate transitions).
- **Code Optimization & Edge-Case Auditing:** Double-checking array boundary indexing for the custom zero-padding implementation in `problem6.m` to ensure that no built-in filtering routines were inadvertently called.

2. Statement of Independent Authorship and Verification

The student explicitly affirms the following principles of academic integrity:

- **Independent Derivations:** Every mathematical proof, analytical justification, discrete metric analysis, and linear system derivation contained in this document was independently constructed, audited, and verified by the student.
- **Original Code Implementation:** The algorithmic blueprints and actual MATLAB code within the designated `Coding zone` blocks were independently written by the student. The resulting images (`sharpened_K4.png`, `sharpened_K8.png`, and `hidden_watermark.png`) were genuinely generated from deterministic script executions.
- **Scholarly Responsibility:** The student takes sole, full academic responsibility for the factual integrity, mathematical correctness, and computational reproducibility of all submitted materials.

Student Name:	Lin Anfan (Norman)
Student ID:	20861656
Program / Major:	BSc in Mathematics (Computer Science Track)
Department:	Department of Mathematics & Dept. of Computer Science and Engineering
Institution:	The Hong Kong University of Science and Technology
Academic Term:	Fall Semester 2026-27
Date of Submission:	October 7, 2026
Author Signature:	*Lin Anfan (Norman)*